Updates to EVM Portal V2 and EVM PYUSDx Portal

MO

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Aug 17, 2026 - Aug 18, 2026

SOURCE CODE
- PYUSDx
- m-portal-v2
- m-portal-v2
- m-portal-v2
- m-portal-v2

ASSESSED COMMIT ID
- 84daa29
- 6193bf0
- 620b84c
- 6f305a2
- f18d446

SEVERITY BREAKDOWN

3	1	2	0
All Findings	Solved	Acknowledged	Risk Accepted

Critical	0	
High	0	
Medium	0	
Low	0	
Informational	3	Solved 1 Ack. 2

INTRODUCTION

M0 engaged Halborn to perform a security assessment of their smart contracts starting and ending on August 18th, 2026. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The M0 Portal V2 codebase in scope consists of smart contracts implementing a cross-chain messaging system for bridging $M tokens and propagating protocol state (earning index, registrar

values) across EVM chains via a hub-and-spoke architecture with pluggable bridge adapters for Hyperlane, Wormhole, and LayerZero.

ASSESSMENT SUMMARY

`Halborn` was allocated `1` day for this engagement and assigned `1` full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, `Halborn` identified several areas for improvement to reduce the likelihood and impact of security risks, which were addressed by the `M0` team. The main recommendations are:

- `Guard SpokePortal._transferAndUnwrap() with an explicit check that IMTokenLike(mToken).isEarning(address(this)) == false, reverting with a distinct error (e.g., SpokePortalIsEarning()) so the misconfiguration surfaces as a loud, single-transaction failure at the point of misconfiguration rather than as a silent DoS on every user's next bridge.`
- `Update the NatSpec of any external-facing helper that constructs permit_ to explicitly document the expected encoding as M0's abi.encode(owner, spender, value, deadline, signature) layout, and to note that standard EIP-2612 (v, r, s) signatures are not accepted directly and must be repacked as a 65-byte signature blob before calling.`
- `Track which operator set the current delegate so _revokeRole can distinguish a compromise revocation from a routine rotation.`

TEST APPROACH AND METHODOLOGY

`Halborn` conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual

testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.

The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts (`Foundry`).
- Fork testing against main networks (`Foundry`).
- Static security analysis of scoped contracts, and imported functions (Slither).

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

REPOSITORIES

(a) Repository: PYUSDx

(b) Assessed Commit ID: 84daa29ad33ff5d495c342db23ee977846ef998d

(c) Items in scope:

- src/portal 2 files
  - interfaces 1 file
    - IPortal.sol Solidity
  - Portal.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

(a) Repository: m-portal-v2

(b) Assessed Commit ID: 6193bf0db3f5633277efc9b9999faeb4faeec594

(c) Items in scope:

- evm/src 5 files
  - bridgeAdapters 3 files
    - layerZero 1 file
      - LayerZeroBridgeAdapter.sol Solidity
    - wormhole 1 file
      - WormholeBridgeAdapter.sol Solidity
    - BridgeAdapter.sol Solidity
  - interfaces 1 file
    - IPortal.sol Solidity
  - Portal.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

(a) Repository: m-portal-v2

(b) Assessed Commit ID: 620b84cd4c3fbce71aed16fee56b243352b2604a

(c) Items in scope:

- evm/src 2 files
  - interfaces 1 file
    - IPortal.sol Solidity
  - Portal.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

(a) Repository: m-portal-v2

(b) Assessed Commit ID: 6f305a22fbcf37de409bf7148b9a41f78353abbc

(c) Items in scope:

- evm/src 2 files
  - HubPortal.sol Solidity
  - Portal.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

(a) Repository: m-portal-v2

(b) Assessed Commit ID: f18d446fb97f1d93d2c0ccc201e223f975317d01

(c) Items in scope:

- evm/src 2 files
  - interfaces 1 file
    - IHubPortal.sol Solidity
  - HubPortal.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:

0521dc49ae8a739c7575a492a967154cd4cfcd43

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Portal.sendTokenWithPermit() NatSpec does not disclose that permit_ is MO's non-standard permit encoding	Informational	1.7	SOLVED 08/24/2026
HAL-002	Unconditional LayerZero delegate clearance in _revokeRole silently wipes delegate on any OPERATOR_ROLE revocation	Informational	1.7	ACKNOWLEDGED 08/24/2026
HAL-003	Strict actualAmount check in SpokePortal._transferAndUnwrap() creates a single-flip DoS on any earner misconfiguration	Informational	0.3	ACKNOWLEDGED 08/24/2026

Findings & Tech Details

HAL-001 SOLVED

Portal.sendTokenWithPermit() NatSpec does not disclose that permit_ is M0's non-standard permit encoding

● Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `Portal.sendTokenWithPermit()` function accepts a `bytes calldata permit_` parameter that is decoded and forwarded to the source token's permit entry point along the bridging path. However, the NatSpec on `sendTokenWithPermit()` documents `permit_` only as an opaque bytes blob and does not state which encoding the Portal expects. In the M0 ecosystem, $M, wM, and mUSD implement the M0-flavored permit signature `permit(address owner, address spender, uint256 value, uint256 deadline, bytes signature)` rather than the EIP-2612 canonical form `permit(address owner, address spender, uint256 value, uint256 deadline, uint8 v, bytes32 r, bytes32 s)`. Integrators who bring a non-M0 permit-supporting token as a bridging source token, or who construct the calldata client-side from a generic EIP-2612 signing helper, will produce a (v, r, s)-encoded blob.

RECOMMENDATION

Update the NatSpec of any external-facing helper that constructs `permit_` to explicitly document the expected encoding as M0's `abi.encode(owner, spender, value, deadline, signature)` layout, and to note that standard EIP-2612 (v, r, s) signatures are not accepted directly and must be repacked as a 65-byte signature blob before calling.

REMEDIATION COMMENT

Addressed in: 052 1dc49ae8a739c7575a492a967154cd4cfcd43

SOLVED: The `M0` team solved the issue in the specified commit id by following the mentioned recommendation.

HAL-002 ACKNOWLEDGED

Unconditional LayerZero delegate clearance in _revokeRole silently wipes delegate on any OPERATOR_ROLE revocation

● Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

`LayerZeroBridgeAdapter._revokeRole()` is responsible for cleaning up LayerZero endpoint authority when an operator is removed. The LayerZero delegate is the address the endpoint recognises as authorised to perform OApp-level recovery and configuration operations, `setConfig()`, `skip()`, `clear()`, `setSendLibrary()`, `setReceiveLibrary()`, and `nilify()`. Without a valid delegate, none of these can be invoked and stuck or malformed bridge messages cannot be recovered in-band.

PR #73 (commit `b36cd10`) removed the conditional guard that previously checked whether the account being revoked was actually the current LZ delegate before clearing it.

The intent behind PR #73 was legitimate, preventing a compromised operator from retaining LZ authority through a third-party delegation after their role is revoked. However, the fix is too broad. Because `AccessControlUpgradeable` supports multiple concurrent `OPERATOR_ROLE` holders, and because `setDelegate()` requires only `OPERATOR_ROLE` (any operator can update the delegate at any time), the revocation of any operator, including ones entirely unrelated to the current delegate, now unconditionally wipes the endpoint delegate. No storage records which operator set the current delegate, so `_revokeRole` cannot distinguish revoking the operator who authorised the delegate from revoking an unrelated one.

RECOMMENDATION

Track which operator set the current delegate so `_revokeRole` can distinguish a compromise revocation from a routine rotation.

REMEDIATION COMMENT

ACKNOWLEDGED: The `M0` team made a business decision to acknowledge this finding and not alter the contracts, stating that:

> Acknowledged. While AccessControlUpgradeable technically permits multiple OPERATOR_ROLE holders, the protocol's operational model assigns this role to a single address. The unconditional delegate removal is therefore intentional. Revoking the sole operator must also remove any LayerZero delegate it may have configured, including a third-party delegate, so that a compromised operator cannot retain indirect authority. Tracking which operator configured the delegate would

> add storage and lifecycle complexity for a multi-operator configuration that is outside the intended deployment model. If multiple operators were ever configured temporarily, clearing the delegate remains fail-safe, and an active operator could restore the intended delegate through setDelegate().

HAL-003 ACKNOWLEDGED

Strict actualAmount check in SpokePortal._transferAndUnwrap() creates a single-flip DoS on any earner misconfiguration

● Informational · 0.3 AO:S/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The `SpokePortal._transferAndUnwrap()` computes `actualAmount = _mBalanceOf(this) - mBalanceBefore` after the source token is unwrapped through `SwapFacility.swapOutM()`, then reverts with `InsufficientAmountReceived` if `actualAmount < specifiedAmount`. On the Hub side, the sibling override tolerates up to `idx/1e12 + 1` wei to absorb the well-known `principal = amount * 1e12 / index` rounding from MToken, but this tolerance was not restored on `SpokePortal`.

The omission is justified off-chain by the PR description's claim that "SpokePortals are not $M earners and must receive the exact amount," but that invariant is not enforced anywhere in the contract. The attacker is any protocol operator, governance delegate, or Registrar-privileged account able to add `SpokePortal` to the earner list on a Spoke chain, whether intentionally (to earn yield on locked $M) or accidentally via a Registrar update. Once `SpokePortal` is earning, `SpokeMToken` credits any incoming transfer as `principal = amount * 1e12 / index` (rounded down), so the portal's balance grows by `amount - 1` wei instead of `amount`. This trips the strict `actualAmount < specifiedAmount` branch on every subsequent `sendToken()` that routes through the unwrap path, permanently reverting outbound bridging on that Spoke chain until `SpokePortal` is unregistered from earners. The same failure mode is also reachable without any earner misconfiguration, since any Spoke-side extension (wM, mUSD, or a future one) that uses indexed accounting in its unwrap-to-$M output can independently return 1 wei short.

RECOMMENDATION

Guard `SpokePortal._transferAndUnwrap()` with an explicit check that `IMTokenLike(mToken).isEarning(address(this)) == false`, reverting with a distinct error (e.g., `SpokePortalIsEarning()`) so the misconfiguration surfaces as a loud, single-transaction failure at the point of misconfiguration rather than as a silent DoS on every user's next bridge.

Alternatively, override `SpokePortal._revertIfInsufficientMReceived()` with the same `idx/1e12 + 1` wei tolerance formula used on `HubPortal`, making the Spoke behavior symmetric with the Hub and eliminating dependence on the off-chain invariant altogether.

REMEDIATION COMMENT

ACKNOWLEDGED: The `M0` team made a business decision to acknowledge this finding and not alter the contracts, stating that:

> Acknowledged. We do not plan to make this change.
> A SpokePortal is intentionally never configured as an $M earner.
> The strict actualAmount >= specifiedAmount check is intentional and preserves the invariant that the Portal receives the full amount before bridging.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.